

PROMPT IA 14

Quality Engineering + Functional BDD

TDD - Testcontainers - Contracts - Cucumber/Gherkin - Godog - Jira - Playwright - k6 - Chaos - Security

Mission : industrialiser une strategie de qualite complete et traçable, du besoin Jira jusqu'aux preuves de release, avec des quality gates automatiques dans Tekton.

test-first where valuable - behavior-driven - traceable - reproducible - evidence-based

1. Role de l'IA

Tu es Principal Quality Engineer / SDET. Implemente les frameworks, tests, fixtures, environnements, rapports, integrations Jira et quality gates necessaires pour backend, frontend, contrats, infrastructure, securite, performance et resilience.

Ne cherche pas un pourcentage de couverture artificiel. La priorite est la couverture des invariants, comportements metier, contrats, risques et parcours critiques. Un test flaky est un defect a corriger.

2. Chaine officielle de qualite

```
Jira Epic / Story / Bug
-> Acceptance Criteria
-> Cucumber/Gherkin .feature
-> Godog (Go)
-> Unit/TDD + Integration + Contracts
-> Functional BDD
-> Playwright E2E
-> k6 / Security / Chaos
-> JUnit + Cucumber reports
-> Evidence de release / Jira
```

Jira gere le travail et la tracabilite. Cucumber/Gherkin est le standard de specification BDD. Godog reste le moteur d execution principal cote Go. Ne pas ajouter un second runtime Cucumber cote Go.

3. Stack standardisee

Couche	Outils	But
Projet / QA	Jira	Epics, Stories, Bugs, AC, releases, tracabilite.
BDD spec	Cucumber / Gherkin	Given / When / Then portable.
BDD Go	Godog	Execution des fichiers .feature.
Unit/TDD Go	testing + testify cible	Domain, use cases, invariants.
Integration	testcontainers-go	PostgreSQL, Redis, Kafka, RabbitMQ.
REST contracts	OpenAPI 3.1 + Schemathesis	Compatibilite et tests generatifs.
gRPC	Buf lint + buf breaking	Contrats Protobuf.
Kafka	Buf + Apicurio	Schemas evenements Protobuf.
Frontend	Vitest + Testing Library	Logique et composants.
E2E	Playwright	Storefront/Admin, navigateurs reels.
Accessibility	axe-core + Playwright	WCAG automatise + clavier.
Visual	Playwright screenshots	Regression Design Packs NOMA.
Performance	k6 + Lighthouse CI	Charge, SLO, frontend.
Chaos	Chaos Mesh	Resilience Kubernetes/multi-site.
Security	Semgrep, gosec, Trivy, Gitleaks, ZAP	SAST/SCA/secrets/DAST.
Infra/Data	terraform test, tfint, helm lint, Kyverno, dbt test	IaC, policies, data.

4. Jira comme source de tracabilite projet / QA

Jira est retenu pour les Epics, User Stories, Bugs, criteres d acceptation, releases et liens vers les executions CI. Les criteres d acceptation doivent etre testables et, pour les comportements critiques, traduits en scenarios Gherkin.

```
JIRA-421 - Refund partiel
Acceptance Criteria
-> montant <= remboursable
-> idempotence
-> audit
-> evenement publie
```

```
Linked feature:
tests/bdd/features/payment/refund.feature
```

Ne pas dupliquer toute la specification dans Jira et Git. Jira porte le besoin et les AC; Git porte les fichiers .feature executables et versionnes.

5. Cucumber/Gherkin comme standard BDD

Utiliser Feature, Background, Scenario, Scenario Outline, Examples, Given, When, Then et And avec parcimonie. Les scenarios doivent decire le comportement metier, pas des clics UI ni des details d implementation.

```
Feature: Inventory reservation
```

```
Scenario: Last unit can only be reserved once
Given SKU NOMA-001 has 1 unit available
When 2 customers reserve the SKU concurrently
Then exactly 1 reservation succeeds
And inventory never becomes negative
```

6. Godog cote Go

Godog execute les fichiers Gherkin contre les use cases/services. Factoriser World/context, fixtures et steps generiques sans creer une couche magique qui masque le domaine.

```
tests/bdd/
features/
checkout/
pricing/
inventory/
payment/
returns/
shipping/
fraud/
rbac/
saga/
steps/
support/
```

7. Domaines BDD prioritaires

- Checkout et confirmation backend.
- Pricing, promotions et coupons.
- Inventory et concurrence de reservation.
- Payment, capture, idempotence et refund.
- Returns et decision de restock/remboursement.
- Shipping et transitions logistiques.
- RBAC/permissions et refus backend.
- Fraud/Risk allow-review-deny.
- Sagas critiques, compensations, timeout et reprise.

8. Scenarios positifs, negatifs et limites

Chaque feature critique doit couvrir happy path, refus, erreur provider, duplicate/replay, timeout, donnees invalides et limites pertinentes.

```
Given a payment was already refunded
When the same idempotency key is submitted 10 times
Then only one refund exists
And only one durable business effect is emitted
```

9. TDD backend

Appliquer RED -> GREEN -> REFACTOR pour les invariants critiques. Les tests unitaires restent plus rapides et plus fins que BDD; ne pas convertir chaque test unitaire en scenario Gherkin.

```
RED -> expected behavior fails
GREEN -> minimal correct implementation
REFACTOR -> clean/factorize
VERIFY -> full impacted suite
```

10. Integration / Testcontainers

Tester PostgreSQL, Redis, Kafka et RabbitMQ avec de vrais services ephemeres, migrations reelles et isolation des donnees. Couvrir transactions, Outbox, consumers, retries et DLQ.

11. Contract testing

REST: OpenAPI 3.1 + Schemathesis. gRPC: Buf lint/breaking. Kafka: Protobuf + Buf + Apicurio compatibility. Aucun breaking change silencieux.

12. Playwright E2E

Playwright teste les parcours navigateur complets Storefront et Admin. Il ne remplace pas BDD: BDD valide le comportement metier; Playwright valide l integration utilisateur/systeme.

```
PR: Chromium critical smoke
Merge: critical E2E
Nightly/pre-prod: Chromium + Firefox + WebKit + mobile + desktop
```

12.1 Storefront

```
Home -> Search -> Product -> Cart -> Login -> Checkout
-> Payment sandbox -> backend confirmation -> Order -> Tracking -> Return
```

12.2 Admin

```
Admin login -> RBAC -> Product/Pricing -> Inventory adjustment
-> Order -> Refund -> Return -> Fraud review -> Audit
```

13. Visual regression et accessibilite

Utiliser Playwright screenshots pour les ecrans critiques NOMA. Utiliser axe-core + tests clavier. Une baseline visuelle ne doit etre mise a jour qu apres review explicite.

14. Performance

k6 valide REST, gRPC et workflows representatifs. Lighthouse CI protege les budgets frontend. Les seuils projet restent: REST p95 < 300 ms, gRPC p95 < 100 ms, Search p95 < 250 ms et Kafka lag normal < 30 s.

```
REST : 500 RPS nominal / 2 000 peak
gRPC : 1 000 req/s nominal / 5 000 peak
Kafka : 1 000 events/s nominal / 5 000 peak / 10 000 burst
```

15. Security testing

```
Semgrep + gosec -> source
Gitleaks -> secrets
Trivy -> deps / OCI / IaC
OWASP ZAP -> DAST
Go fuzzing -> parsers / webhooks / validation
Kyverno tests -> Kubernetes policies
```

Ajouter des scenarios BDD de securite metier lorsque le comportement est fonctionnel: permission refusee, webhook replay, double refund, session expiree, etc.

16. Chaos et PRA

Chaos Mesh couvre perte worker, broker Kafka, failover PostgreSQL, latence/perte reseau, coupure WAN, indisponibilite provider et perte d un site. Chaque experience a hypothesis, blast radius, abort conditions et evidence.

17. Formats de rapports

Les resultats doivent rester portables. Generer JUnit XML pour CI et, pour BDD, un format Cucumber exploitable. Conserver traces Playwright, captures sur echec, rapports k6/security/chaos.

```
Godog / BDD
-> JUnit XML
-> Cucumber-compatible report
-> Tekton artifacts
-> Jira reference / release evidence
```

18. Jira et Xray

Jira est obligatoire dans le processus projet/QA. Xray n est pas obligatoire au demarrage. Ne pas coupler les tests a Xray; conserver des formats standards afin de pouvoir l ajouter plus tard sans reecrire les scenarios.

SpecFlow n est pas retenu. Godog reste le runtime Go. Cucumber/Gherkin est le standard de specification, pas un second moteur a installer cote backend.

19. Convention de tracabilite

Permettre de relier un scenario a une Story Jira sans rendre le scenario illisible. Utiliser tags ou metadata simples selon le support du runner.

```
@JIRA-421 @payment @critical
Scenario: Partial refund succeeds ...
```

Ne pas coder de credentials Jira dans les tests. Toute publication automatique de resultats utilise des credentials CI fournis par OpenBao et des permissions minimales.

20. Pipeline Tekton

```
PR
-> lint
-> unit/TDD
-> integration
-> contract tests
-> Functional BDD (Gherkin/Godog)
-> frontend component tests
-> ephemeral environment
-> Playwright smoke
-> security gates
-> OCI build / Trivy / SBOM / Cosign / Harbor
```

La couche BDD s execute apres unit/integration/contracts et avant les smoke E2E Playwright, sauf optimisation explicite par impact analysis.

21. Quality gates

Gate	Bloquant
Unit / invariants critiques	Oui
Integration critique	Oui
Breaking contracts	Oui

Gate	Bloquant
Functional BDD critique	Oui
Playwright critical smoke	Oui
Secrets detectes	Oui
High/Critical security selon policy	Oui
Accessibility critique	Oui
Performance regression majeure	Oui avant prod
Chaos/PRA	Obligatoire pour certification release/PRA

22. Flaky tests

Mesurer le flaky rate. Quarantaine temporaire uniquement avec owner, ticket Jira et date limite. Interdire les retries illimites qui masquent une suite instable.

23. Test data

Factories/fixtures deterministes, donnees synthetiques/anonymes, horloge controlee et providers sandbox. Aucun dump production brut. Les montants financiers utilisent minor units et cas d arrondi explicites.

24. Environnements

```

LOCAL -> unit + targeted integration + BDD cible
PR -> unit + contracts + integration + BDD + smoke E2E
MERGE -> full critical suites + security
NIGHTLY -> browser matrix + extended BDD/security/perf
PRE-PROD -> full E2E + k6 + ZAP + Chaos + restore
PROD -> smoke non destructif + RUM/SLO

```

25. Definition of Done

- Jira porte Epics, Stories, Bugs, AC et releases.
- Les comportements critiques sont specifiques en Cucumber/Gherkin Given/When/Then.
- Godog execute les .feature cote Go.
- Les scenarios BDD critiques sont bloquants dans Tekton.
- Les rapports BDD sont exportes en formats JUnit/Cucumber portables.
- Xray peut etre ajoute plus tard sans reecriture des tests.
- Unit, integration, contracts, BDD et E2E restent des couches distinctes.
- Storefront/Admin disposent de Playwright, axe et visual regression.
- k6, security, Chaos et restore produisent des preuves de release.
- Chaque release critique relie commit, digest OCI, resultats de tests et reference Jira.

Le livrable est termine lorsque le besoin Jira peut etre relie a des criteres d acceptation, a des scenarios executables, a des resultats CI et a une preuve de release - sans transformer Jira, Gherkin ou Xray en dependance technique du domaine.